《算法设计与分析》实验报告

实验五　分支限界法

报告书			
姓名	顾凯琪	指导教师	方贤忠
学号	226001624	日　　期	2025/05/28
班级	22 级计算机 2 班		

实验内容

[要求先在 OJ 上提交通过]

1、[OJ]分支限界 1

2、[OJ]分支限界-堂练

实验目的

1）掌握分支限界法的应用

2）学习掌握本类算法的设计与实现，在 OJ 上提交算法代码并测试通过，完成实验报告书写。

【如有查重雷同，请在此说明】

代码独立完成，无雷同。

【如有 OJ 未完成，请在此说明，并在本实验文档中补全】

自行完善：

OJ 实验 1 作业正常时间完成。

OJ 实验 2 堂练全部完成。

实验过程和步骤

Module 1: 分支限界 1

【第一部分】

【1、填格子 4】

填格子4

Description

有一个由数字 0、1 组成的方阵中，存在一任意形状的封闭区域，封闭区域由数字1 包围构成，每个节点只能走上下左右 4 个方向。现要求把封闭区域内的所有空间都填写成2 .例如： 6x6 的方阵：

6

0 0 0 0 0 0

0 0 1 1 1 1

0 1 1 0 0 1

1 1 0 0 0 1

1 0 0 0 0 1

1 1 1 1 1 1

填写后如下：

0 0 0 0 0 0

0 0 1 1 1 1

0 1 1 2 2 1

1 1 2 2 2 1

1 2 2 2 2 1

1 1 1 1 1 1

Input

每组测试数据第一行一个整数 n($1 \leq n \leq 30$)

接下来 n 行，由 0 和 1 组成的 nxn 的方阵。

封闭区域内至少有一个0 。

Output

已经填好数字 2 的完整方阵。注意矩阵的每个数字之间都有一个空格.

Sample Input 1

```
6
0 0 0 0 0 0
0 0 1 1 1 1
0 1 1 0 0 1
1 1 0 0 0 1
1 0 0 0 0 1
1 1 1 1 1 1
```

Sample Output 1

```
0 0 0 0 0 0
0 0 1 1 1 1
0 1 1 2 2 1
1 1 2 2 2 1
1 2 2 2 2 1
1 1 1 1 1 1
```

【2、程序代码及注释】

```
#include <bits/stdc++.h>
using namespace std;

int map1[31][31] = {0};                 // 地图矩阵，记录输入的地图信息（0 表示空地，1 表示墙）
bool visited[31][31] = {false};         // 标记某个位置是否被访问
int n;                                  // 地图的尺寸
int dir[4][2] = {{1,0},{-1,0},{0,1},{0,-1}}; // 四个方向：下、上、右、左

// 广度优先搜索，用于从某点(x, y)出发标记所有连通的0 区域
void bfs(int x, int y) {
    queue<pair<int, int>> q;
    visited[x][y] = true;
    q.push({x, y});

```

```
    while (!q.empty()) {
        auto [cx, cy] = q.front();
        q.pop();

        for (int d = 0; d < 4; d++) {
            int nx = cx + dir[d][0];
            int ny = cy + dir[d][1];

            // 边界判断 + 未访问 + 是空地（0）
            if (nx >= 1 && nx <= n && ny >= 1 && ny <= n &&
                map1[nx][ny] == 0 && !visited[nx][ny]) {
                visited[nx][ny] = true;
                q.push({nx, ny});
            }
        }
    }
}

int main() {
    cin >> n;

    // 输入地图，并标记墙体（1）为已访问
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cin >> map1[i][j];
            if (map1[i][j] == 1)
                visited[i][j] = true;
        }
    }

    // 从边缘两行（第 1 行和第 n 行）向内扩展
    for (int i = 1; i <= n; i += n - 1) {
        for (int j = 1; j <= n; j++) {
            if (!visited[i][j])
                bfs(i, j);
        }
    }

    // 从边缘两列（第 1 列和第 n 列）向内扩展
    for (int i = 1; i <= n; i += n - 1) {
        for (int j = 1; j <= n; j++) {
            if (!visited[j][i])
                bfs(j, i);
```

```
        }
    }

    // 输出结果：已访问区域保持原值，未访问空地区域标记为2
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (visited[i][j])
                cout << map1[i][j] << " ";
            else
                cout << "2 ";
        }
        cout << endl;
    }

    return 0;
}
```

【3、遇到的问题及解决方法】

Time Limit Exceed:刚开始使用深度搜索，时间复杂度过高，导致节点多次遍历产生超时。并且没有搞清边界范围，导致一直产生错误。

【4、算法分析与知识点】

该程序旨在识别一个二维地图中所有被墙体完全包围的空地区域，并将这些区域用数字 2 替换。地图以 n × n 的二维数组形式输入，其中数字 1 表示墙体，0 表示空地。通过判断空地是否与地图的边缘连通来识别“封闭区域”，被封闭的 0 会被替换为 2，其余区域保持原状。

程序的核心算法是广度优先搜索（BFS）。首先将所有墙体（1）标记为已访问。接着，从地图的四条边界（第一行、最后一行、第一列、最后一列）出发，对尚未访问的空地进行 BFS 遍历，所有可以从边缘到达的空地都会被标记为“已访问”，意味着这些空地与边界连通，不能算作被封闭区域。最后，再遍历整张地图，对所有未被访问且值为 0 的位置赋值为 2，从而完成封闭区域的识别与替换。

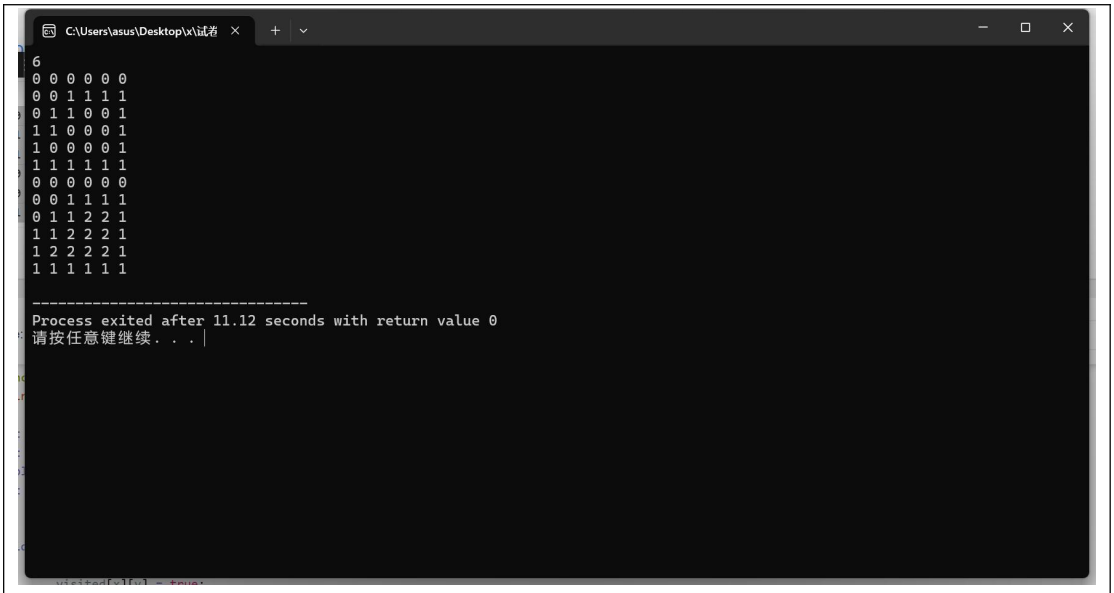
本算法涉及的关键知识点包括图的遍历（BFS）、二维数组的边界处理以及连通块判断。广度优先搜索适用于查找最短路径或判断连通性，其特点是逐层向外扩展，能够有效避免深度优先搜索可能出现的栈溢出问题。

就时间复杂度而言，由于程序最多对每个单元格访问一次，BFS 每个节点只入队和出队一次，因此总的时间复杂度为 $O(n^2)$，其中 n 为地图的边长。至于空间复杂度，主要用于存储地图本身、访问标记数组 visited、以及 BFS 所用的队列，三者的空间开销均为 $O(n^2)$，因此整体空间复杂度也是 $O(n^2)$。

【5、AC 截图及运行测试截图】



2025-5-19 15:51:45	6658167c5c14	Accepted	1000	1ms	3MB	C++	226001624
2025-5-19 15:49:48	a668b44a8ace	Wrong Answer	1000	1ms	3MB	C++	226001624
2025-5-19 15:45:26	26efabf46ce5	Wrong Answer	1000	1ms	3MB	C++	226001624



【1、最短路径】



【2、程序代码及注释】

```cpp
#include <bits/stdc++.h>
using namespace std;

// 地图数组、起点终点坐标
int map1[21][21];
int m, n;
int sx, sy, ex, ey;

// 访问标记和步数记录数组
bool visited[21][21];
int step[21][21];

// 定义 BFS 使用的队列和方向数组（上下左右）
queue<pair<int, int>> q;
int dir[4][2] = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
```

```cpp

// 最短路径长度
int maxnum;

void init() {
    while (!q.empty()) q.pop();                 // 清空队列
    maxnum = 999999;                            // 初始化最大值为一个较大的数
    memset(visited, false, sizeof(visited));    // 清空访问标记
    memset(step, 0x3f, sizeof(step));           // 初始化步数为极大值（使用 memset 设为 0x3f3f3f）
}

// 广度优先搜索，寻找从起点到终点的最短路径
void bfs(int x, int y) {
    visited[x][y] = true;
    step[x][y] = 0;
    q.push({x, y});

    while (!q.empty()) {
        auto cur = q.front();
        q.pop();
        for (int i = 0; i < 4; i++) {
            int newx = cur.first + dir[i][0];
            int newy = cur.second + dir[i][1];

            // 如果走到了终点
            if (newx == ex && newy == ey) {
                maxnum = step[cur.first][cur.second] + 1;
                return;
            }

            // 边界检测 + 障碍检测 + 是否访问过
            if (newx < 0 || newx >= m || newy < 0 || newy >= n || visited[newx][newy] || map1[newx][newy] == -1)
                continue;

            visited[newx][newy] = true;
            step[newx][newy] = step[cur.first][cur.second] + 1;
            q.push({newx, newy});
        }
    }
}
```

```
56.
57.// 输出路径图，标记起点a、终点b、墙体-1、无法到达区域为0，其余显示步数
58.void show() {
59.    for (int i = 0; i < m; i++) {
60.        for (int j = 0; j < n; j++) {
61.            if (i == sx && j == sy)
62.                cout << setw(3) << "a";
63.            else if (i == ex && j == ey)
64.                cout << setw(3) << "b";
65.            else if (map1[i][j] == -1)
66.                cout << setw(3) << -1;
67.            else if (step[i][j] >= maxnum)
68.                cout << setw(3) << 0;
69.            else
70.                cout << setw(3) << step[i][j];
71.        }
72.        cout << endl;
73.    }
74.    cout << endl;
75.}
76.
77.int main() {
78.    // 支持多组输入
79.    while (cin >> m >> n) {
80.        cin >> sx >> sy >> ex >> ey;
81.        for (int i = 0; i < m; i++) {
82.            for (int j = 0; j < n; j++) {
83.                cin >> map1[i][j];
84.            }
85.        }
86.        init();
87.        bfs(sx, sy);
88.        show();
89.    }
90.    return 0;
91.}
```

【3、遇到的问题及解决方法】

格式错了。。本题每行后不带空格

【4、算法分析与知识点】

本程序旨在解决一个典型的迷宫最短路径问题，采用了广度优先搜索（BFS）算法来寻找从起点 (sx, sy) 到终点 (ex, ey) 的最短路径。输入是一个 m × n 的二维地图，其中 -1 表示障碍不可通行，其余为可通行区域。BFS 的核心思想是从起点出发，利用队列结

构逐层扩展四个方向（上、下、左、右），每次移动一步，并使用 step 数组记录每个点距离起点的最小步数。一旦终点被访问，即可得出最短路径长度。搜索过程中，为避免重复访问和死循环，程序还使用 visited 数组标记已访问的点。最终，程序通过 show() 函数以格式化输出展示路径状态，起点显示为 a，终点为 b，不可通达点显示为 0。

该程序主要考查了图的遍历算法（BFS）、队列的使用、二维数组边界控制、以及最短路径问题中的状态转移逻辑。其中 bfs() 函数是算法核心，队列的 push 和 pop 操作结合 step 的更新完成路径长度计算，而 visited 数组确保了每个点最多访问一次。整个 BFS 的过程在最坏情况下会遍历所有非障碍点。

从复杂度角度分析，设地图大小为 m × n，即最多有 m × n 个点。每个点最多进队一次，四个方向尝试最多访问四次，因此时间复杂度为 O(m × n)。而空间方面，使用了 map1、visited 和 step 三个大小为 m × n 的二维数组，以及一个最多存储 m × n 个元素的队列，因此空间复杂度同样为 O(m × n)。在空间换时间的策略下，该程序能有效处理中等规模的地图搜索问题，具有较强的实用性和普适性。

【5、AC 截图及运行测试截图】

2025-5-19 16:49:35	a48e07567251	Accepted	1001	2ms	3MB	C++	226001624
2025-5-19 16:42:47	addfbf677984	Wrong Answer	1001	2ms	3MB	C++	226001624
2025-5-19 16:32:04	c5a14742ad7b	Wrong Answer	1001	2ms	3MB	C++	226001624
2025-5-19 16:28:22	f10841f5c54c	Wrong Answer	1001	3ms	3MB	C++	226001624
2025-5-19 16:00:47	31c034eeab0c	Wrong Answer	1001	0ms	3MB	C++	226001624
2025-5-19 15:56:25	2e086d55b629	Wrong Answer	1001	3ms	3MB	C++	226001624

```
5 1 5
3 3 1 2 5
3

--------------------------------
Process exited after 5.251 seconds with return value 0
请按任意键继续. . .
```

【1、藏宝图 2-彼岸花】

藏宝图2-彼岸花

Description

某人发现了一张藏宝图，由于价值巨大，因此决定拼了命也要试一下，毕竟人生难得几回搏；

藏宝区域是一个m*n的二维空间，有一个入口，一个藏宝点；藏宝点同时也是唯一的出口；

现在给你藏宝图，但是藏宝图被黑心的卖家卖了n份，入口在某一个确定的时间才会开启；

因此你必须不计代价，以最短的距离冲向藏宝点，才能获得宝藏，前提当然是还有一口气；

藏宝图的符号表示如下：

!表示起点位置，+表示宝藏的位置。

*代表通路不损耗生命值，w代表此处损耗生命值-1。

该空间有可能会出现彼岸花，用e表示；如果有幸得到彼岸花，生命值会恢复到初始值。

Input

输入的第一行包含三个整数：m，n，t。代表m行n列的地图和生命值t。m,n都是小于200的正整数，t是小于10的正整数，第2行开始的m行是m行n列的地图，其中!代表起点，+表示藏宝点。*代表通路，w代表地刺。

Output

如果能活着到达，输出达到的最少步数。如果无法到达，则输出-1。

Sample Input 1

```
6 7 2
w!wwwww
*www***
*ww+*w*
*wwwww*
*wwwww*
*******
```

Sample Output 1

```
20
```

【2、程序代码及注释】

```
#include <bits/stdc++.h>
using namespace std;

// 定义人物结构体，记录当前位置(x,y)、剩余生命值(hp)、已走步数(step)
class People {
public:
    int x;
    int y;
    int hp;
    int step;
};

int m, n, t;                          // 地图大小 m×n，初始生命值 t
char map1[201][201];                  // 地图信息
queue<People> q;                      // BFS 队列
int dir[4][2] = {{-1,0},{1,0},{0,1},{0,-1}}; // 四个方向：上、下、右、左
int maxstep = 999999999;              // 记录最短路径
bool visited[201][201][10] = {false}; // 三维数组，记录在某坐标某生命值是否访问过

// BFS 搜索函数
void bfs(int x, int y, int hp) {
    People p = {x, y, t, 0};      // 初始化起点
    visited[p.x][p.y][p.hp] = true;
    q.push(p);
```

```
25.
26.     while (!q.empty()) {
27.         p = q.front();
28.         q.pop();
29.
30.         // 如果当前是终点 '+'
31.         if (map1[p.x][p.y] == '+') {
32.             if (p.step < maxstep)
33.                 maxstep = p.step;
34.             return;
35.         }
36.
37.         // 尝试四个方向移动
38.         for (int i = 0; i < 4; i++) {
39.             People temp;
40.             temp.x = p.x + dir[i][0];
41.             temp.y = p.y + dir[i][1];
42.             temp.hp = p.hp;
43.
44.             // 越界处理
45.             if (temp.x < 0 || temp.x >= m || temp.y < 0 || temp
    .y >= n)
46.                 continue;
47.
48.             // 遇到补给点 e：生命值重置
49.             if (map1[temp.x][temp.y] == 'e')
50.                 temp.hp = hp;
51.
52.             // 遇到陷阱 w：生命值 -1
53.             if (map1[temp.x][temp.y] == 'w')
54.                 temp.hp -= 1;
55.
56.             // 若还活着且未访问，继续探索
57.             if (temp.hp >= 1 && !visited[temp.x][temp.y][temp.h
    p]) {
58.                 temp.step = p.step + 1;
59.                 visited[temp.x][temp.y][temp.hp] = true;
60.                 q.push(temp);
61.             }
62.         }
63.     }
64. }
65.
```

```cpp
int main() {
    cin >> m >> n >> t;

    // 读入地图
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            cin >> map1[i][j];

    // 找到起点 '!' 并启动 BFS
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            if (map1[i][j] == '!')
                bfs(i, j, t);

    cout << maxstep << endl;
    return 0;
}
```

【3、遇到的问题及解决方法】

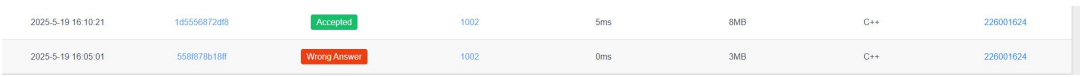
无

【4、算法分析与知识点】

该程序使用广度优先搜索（BFS）算法，从地图中所有起点 ！出发，寻找通往终点 + 的最短路径。地图中的每一个位置既受到体力限制，也可能通过 e 点恢复体力或在 w 点消耗体力，因此程序引入了三维访问数组 visited[x][y][hp] 来完整表示状态，从而避免重复搜索。每次扩展一个状态时，程序会判断是否越界、是否撞墙或补血，并根据剩余体力决定是否可以继续前进，从而保证搜索的合理性和正确性。通过 BFS 的逐层扩展性质，一旦搜索到终点，其对应的步数即为最短路径。

算法的时间复杂度为 $O(m \times n \times t)$，其中 m 和 n 分别为地图的行列数，t 为初始体力值，因为每个格子可能以不同体力值被访问一次。空间复杂度同样为 $O(m \times n \times t)$，主要由三维数组 visited 和搜索队列 q 决定。由于状态数有限，算法能在合理时间内完成搜索，适用于体力值较小、地图中障碍和补给适度分布的场景。

【5、AC 截图及运行测试截图】





Module 2:　分支限界-堂练

【1、不如走楼梯】

不如走楼梯

Description

有个电梯，每一层楼都可以停，只是算法混乱了，所以你得写个补丁；第i层楼(1<=i<=N)上有一个数字Ki(0<=Ki<=N)，表示上或下的层数（相对于当前层），每层楼都可以上或下。当然，如果不能满足要求（没有的层），相应的按钮就会失灵。例如：3 3 1 2 5代表了Ki（在第一层可以上或下3层；当然下是不可能的，第三层可以上或下1层），从一楼开始。在一楼，按"上"可以到4楼，按"下"是不起作用的，因为没有-2楼。那么，从A楼到B楼至少要按几次按钮?

Input

共二行。

第一行为3个用空格隔开的正整数，表示 N,A,B（层数，开始层，结束层）；(1≤N≤200, 1≤A,B≤N)N,A,B(1≤N≤200,1≤A,B≤N)。

第二行为N个用空格隔开的非负整数，表示每层按钮的数值Ki。

Output

一行，即最少按键次数；若无法到达，则输出-1。

Sample Input 1

```
5 1 5
3 3 1 2 5
```

Sample Output 1

```
3
```

【2、程序代码及注释】

```
#include<bits/stdc++.h>
using namespace std;

int n, a, b, k[210];
bool visited[210] = { false };

void bfs() {
    queue<pair<int, int> > q;
    pair<int, int> p, t;
    p.first = a; p.second = 0;
    q.push(p);
    while (!q.empty()) {
```

```
        p = q.front(); q.pop();
        if (visited[p.first])
            continue;
        if (p.first == b) {
            cout << p.second << endl;
            return;
        }
        visited[p.first] = true;
        if (p.first - k[p.first] > 0) { // 向下按
            t.first = p.first - k[p.first];
            t.second = p.second + 1;
            q.push(t);
        }
        if (p.first + k[p.first] <= n) { // 向上按
            t.first = p.first + k[p.first];
            t.second = p.second + 1;
            q.push(t);
        }
    }
    cout << -1 << endl;
}

int main() {
 cin >>n>>a>>b;
 for (int i = 1; i <= n; i++)
        cin >> k[i];
    bfs();
    return 0;
}
```

【1、再填格子】

再填格子

Description

有一个由数字 0、1 组成的方阵中，存在一任意形状的封闭区域，封闭区域由数字1 包围构成，每个节点只能走上下左右 4 个方向。现要求只把最大封闭区域内的空间填写成2 。

例如，6x6 的方阵如下：

6

0 1 0 0 0 0

1 0 1 1 1 1

0 1 1 0 0 1

1 1 0 0 0 1

1 0 0 0 0 1

1 1 1 1 1 1

填写后如下：

0 1 0 0 0 0

1 0 1 1 1 1

0 1 1 2 2 1

1 1 2 2 2 1

1 2 2 2 2 1

1 1 1 1 1 1

Input

每组测试数据第一行一个整数 $n(1 \leq n \leq 30)$.

接下来 n 行，由 0 和 1 组成的 nxn 的方阵.

封闭区域内至少有一个0，测试数据保证最大区域只有一个.

Output

已经填好数字 2 的完整方阵。(每个数字后面有一个空格！)

Sample Input 1 图

```
6
0 1 0 0 0 0
1 0 1 1 1 1
0 1 1 0 0 1
1 1 0 0 0 1
1 0 0 0 0 1
1 1 1 1 1 1
```

Sample Output 1

```
0 1 0 0 0 0
1 0 1 1 1 1
0 1 1 2 2 1
1 1 2 2 2 1
1 2 2 2 2 1
1 1 1 1 1 1
```

【2、程序代码及注释】

```
#include<bits/stdc++.h>
using namespace std;

int n;
int map1[35][35] = {0};
int maxid = -1;
int maxnum = 0;
queue<pair<int,int>> q;
int dir[4][2] = {{-1,0},{1,0},{0,-1},{0,1}};
int visited[35][35] = {0};
int id = 2;

int bfs(int x,int y,int id){
 int cnt = 0;
 pair<int,int> cur;
 cur.first = x;
 cur.second = y;
 visited[x][y] = id;
 q.push(cur);
 while(!q.empty()){
  cur = q.front();
  q.pop();


  for(int i = 0;i<4;i++){
```

```
    pair<int,int> temp;
    temp.first = cur.first+dir[i][0];
    temp.second = cur.second + dir[i][1];
    if(temp.first<0 || temp.first >=n || temp.second<0 || temp.second>=n || visited[temp.first][temp.second] == id || map1[temp.first][temp.second] == 1){
     continue;
    }
    visited[temp.first][temp.second] = id;
    cnt++;
    q.push(temp);

   }
  }
  return cnt;

}


void show(){
  for(int i =0;i<n;i++){
   for(int j = 0;j<n;j++){
    if(visited[i][j] == maxid){
     cout<<"2"<<" ";
    }
    else{
     cout<<map1[i][j]<<" ";
    }
   }
   cout<<endl;
  }
}

int main(){
  cin>>n;
  for(int i =0;i<n;i++){
   for(int j = 0;j<n;j++){
    cin>>map1[i][j];
    if(map1[i][j] == 1) visited[i][j] =1;
   }
  }

  for(int i = 0;i<n;i += n-1){
```

```
  for(int j =0;j<n;j++){
   if(visited[i][j] == 0 && map1[i][j] == 0)
   bfs(i,j,1);
  }
 }

for(int j = 0;j<n;j += n-1){
  for(int i =0;i<n;i++){
   if(visited[i][j] == 0 && map1[i][j] == 0)
   bfs(i,j,1);
  }
 }

for(int i =0;i<n;i++){
  for(int j = 0;j<n;j++){
   if(visited[i][j] == 0){
    int cnt = bfs(i,j,id);
    if(cnt > maxnum){
     maxnum = cnt;
     maxid = id;
    }
    id++;
   }
  }
 }

  show();


}
```

【1、推箱子】



推箱子

Description

码头仓库是划分为 n×m 个格子的矩形阵列。有公共边的格子是相邻格子。当前仓库中有的格子是空闲的；有的格子则已经堆放了沉重的货物。由于堆放的货物很重，单凭仓库管理员的力量是无法移动的。仓库管理员有一项任务，要将一个小箱子推到指定的格子上去。管理员可以在仓库中移动，但不能跨过已经堆放了货物的格子。管理员站在与箱子相对的空闲格子上时，可以做一次推动，把箱子推到另一相邻的空闲格子。推箱时只能向管理员的对面方向推。由于要推动的箱子很重，仓库管理员想尽量减少推箱子的次数。

对于给定的仓库布局，以及仓库管理员在仓库中的位置和箱子的开始位置和目标位置，设计一个解推箱子问题的分支限界法，计算出仓库管理员将箱子从开始位置推到目标位置所需的最少推动次数。

Input

输入第 1 行有 2 个正整数 n 和 m（1<=n,m<=100），表示仓库是 n×m 个格子的矩形阵列。接下来有 n 行，每行有 m 个字符，表示格子的状态。

S 表示格子上放了不可移动的沉重货物；

w 表示格子空闲；

M 表示仓库管理员的初始位置；

P 表示箱子的初始位置；

K 表示箱子的目标位置。

Output

将计算出的最少推动次数输出。如果仓库管理员无法将箱子从开始位置推到目标位置则输出"No solution！"。

Sample Input 1 复制

```
10 12
SSSSSSSSSSSS
SwwwwwwwSSSS
SwSSSSwwSSSS
SwSSSSwwSKSS
SwSSSSwwSwSS
SwwwwwPwwwww
SSSSSSSwSwSw
SSSSSSPwSwww
SSSSSSSSSSSS
SSSSSSSSSSSS
```

Sample Output 1

```
7
```

【2、程序代码及注释】

```
#include <bits/stdc++.h>
using namespace std;
using pii = pair<int,int>;
const int INF = 1e9;
int n, m;
vector<string> grid;
int dx[4] = {-1,1,0,0};
int dy[4] = {0,0,-1,1};

bool in_bounds(int x, int y) {
    return x >= 0 && x < n && y >= 0 && y < m;
}

bool canReach(int sx, int sy, int tx, int ty, int bx, int by) {
    vector<vector<bool>> vis(n, vector<bool>(m, false));
    queue<pii> q;
    vis[sx][sy] = true;
    q.emplace(sx, sy);
    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        if (x == tx && y == ty) return true;
        for (int d = 0; d < 4; ++d) {
            int nx = x + dx[d], ny = y + dy[d];
            if (!in_bounds(nx, ny) || vis[nx][ny]) continue;
            if (nx == bx && ny == by) continue;
            if (grid[nx][ny] == 'S') continue;
```

```
            vis[nx][ny] = true;
            q.emplace(nx, ny);
        }
    }
    return false;
}

int main() {
    cin >> n >> m;
    grid.resize(n);
    for (int i = 0; i < n; i++) cin >> grid[i];

    int sx=-1, sy=-1, bx=-1, by=-1, tx=-1, ty=-1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (grid[i][j] == 'M') { sx = i; sy = j; grid[i][j] = 'w'; }
            else if (grid[i][j] == 'P') { bx = i; by = j; grid[i][j] = 'w'; }
            else if (grid[i][j] == 'K') { tx = i; ty = j; grid[i][j] = 'w'; }
        }
    }

    if (bx == tx && by == ty) {
        cout << 0;
        return 0;
    }

    queue<tuple<int,int,int,int>> q;
    unordered_set<long long> visited;
    auto encode = [&](int bx, int by, int px, int py) {
        return ((long long)bx * m + by) << 20 | (px * m + py);
    };
    q.emplace(bx, by, sx, sy);
    visited.insert(encode(bx,by,sx,sy));
    int pushes = 1;

    while (!q.empty()) {
        int qs = q.size();
        while (qs--) {
            auto [cx, cy, px, py] = q.front(); q.pop();
            if (cx == tx && cy == ty) {
```

```
                cout << pushes;
                return 0;
            }
            vector<vector<bool>> reach(n, vector<bool>(m, false
));
            queue<pii> pq;
            reach[px][py] = true;
            pq.emplace(px, py);
            while (!pq.empty()) {
                auto [x,y] = pq.front(); pq.pop();
                for (int d = 0; d < 4; ++d) {
                    int nx = x + dx[d], ny = y + dy[d];
                    if (!in_bounds(nx, ny) || reach[nx][ny]) co
ntinue;
                    if (nx == cx && ny == cy) continue;
                    if (grid[nx][ny] == 'S') continue;
                    reach[nx][ny] = true;
                    pq.emplace(nx, ny);
                }
            }
            for (int d = 0; d < 4; ++d) {
                int rx = cx - dx[d], ry = cy - dy[d];
                int nx = cx + dx[d], ny = cy + dy[d];
                if (!in_bounds(rx, ry) || !in_bounds(nx, ny)) c
ontinue;
                if (!reach[rx][ry]) continue;
                if (grid[nx][ny] == 'S') continue;
                long long code = encode(nx, ny, cx, cy);
                if (visited.count(code)) continue;
                visited.insert(code);
                q.emplace(nx, ny, cx, cy);
            }
        }
        pushes++;
    }

    cout << "No solution! ";
    return 0;
}
```

【第二部分】

【同类题型实验扩展/可选/会影响分值】

1、题目+测试数据+结果

2、分析

3、建议：难度不低于实验题

【1、藏宝图】

带生命值限制的迷宫最短路径搜索

描述：

给定一个 m×n 的地图，其中包含起点（'!'）、终点（'+'）、通路、障碍（'w'代表地刺）等元素。

从起点出发，寻找到达终点的最短步数。

角色初始拥有一定的生命值 t，每经过一个带刺地（'w'）会消耗 1 点生命值，生命值耗尽则不能继续通行。

请设计一个算法，计算在生命值限制下，从起点到终点的最短路径长度；如果无法到达终点，返回 -1。

【2、程序代码及注释】

```
#include<iostream>
#include <cstring>
using namespace std;
struct people{ //结构体用来存储当前节点的坐标、生命值、步数
 int x,y;
 int life;
 int step;
};
people que[40000]; //定义的队列
int m,n,t;   //代表地图规格大小和生命值
char map[201][201];  //用于存储输入的地图
int dx[4] = {-1,0,1,0}; //下右上左
int dy[4] = {0,-1,0,1};
int visited[201][201][10]; //判断节点是否被访问，节点一共包含x,y,life三个标记值
int bfs(int x,int y){
 memset(visited,0,sizeof(visited));
 int head=0,tail=0;
 que[tail].x = x;  //初始化起始点
 que[tail].y = y;
 que[tail].life = t;
 que[tail++].step = 0;
 visited[x][y][t]=1;  //标记为已访问
 while(head<tail){
  people temp = que[head++]; //出队
  if(map[temp.x][temp.y] == '+') //判断是否为终点
   return temp.step;
  for(int i=0;i<4;i++){  //四个方向遍历
```

```
    int temp_x = temp.x + dx[i];
    int temp_y = temp.y + dy[i];
    if(temp_x>=0 && temp_y>=0 &&temp_x<m && temp_y<n){ //剪枝判断是否超出地图大小
     if(map[temp_x][temp_y]!='w'&& !visited[temp_x][temp_y][temp.life]){  //判断是否为通路且不是已访问点
      que[tail].x = temp_x;
      que[tail].y = temp_y;
      que[tail].life = temp.life;
      que[tail++].step = temp.step+1;  //入队
      visited[temp_x][temp_y][temp.life] = 1;  //标记为访问
     }
     if(map[temp_x][temp_y] =='w' && temp.life > 1){  //若为地刺则要进行生命值的判断
      int temp_t=temp.life-1;       //生命值减一
      if(!visited[temp_x][temp_y][temp_t]){    //判断是否为已访问点
       que[tail].x = temp_x;
       que[tail].y = temp_y;
       que[tail].life = temp_t;
       que[tail++].step = temp.step+1;    //入队
       visited[temp_x][temp_y][temp_t] = 1;  //标记为已访问
      }
          }
    }

   }
 }
 return -1;
}

int main(){
 cin>>m>>n>>t;
 for(int i=0;i<m;i++){
  for(int j=0;j<n;j++){ //地图初始化
   cin>>map[i][j];
  }
 }
 for(int i=0;i<m;i++){  //寻找起点
  for(int j=0;j<n;j++){
   if(map[i][j]=='!')
    cout<<bfs(i,j)<<endl;
  }
```

```
67. }
68. return 0;
69.}
```

【3、遇到的问题及解决方法】

Wrong Answer:生命值为 0 时不能存活，在遇到地刺时应该增加一个生命值大于 1 的限定条件。

【4、算法分析与知识点】

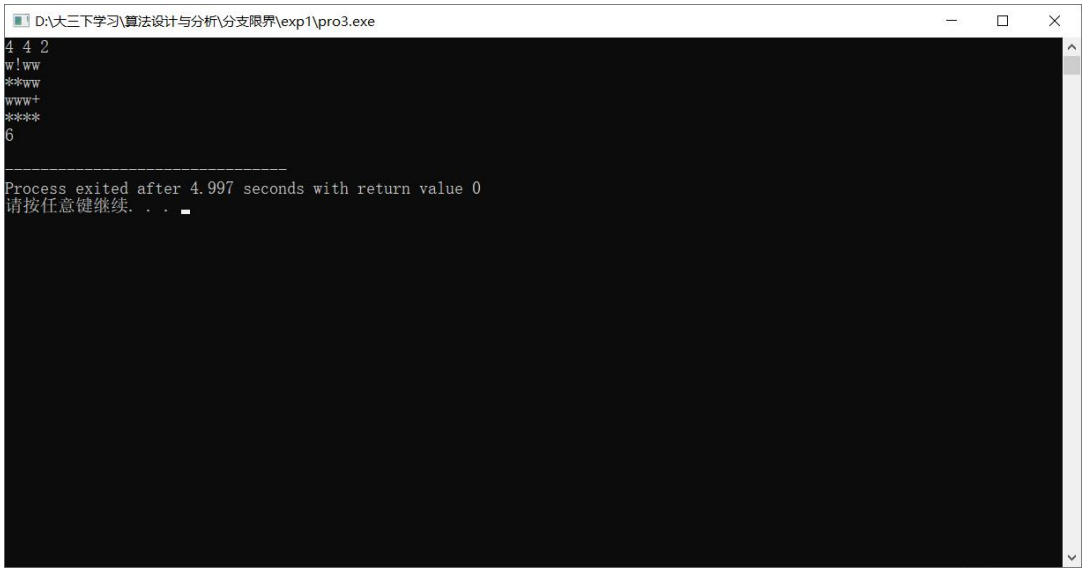
首先我们对题目进行分析，根据题目描述，输入一个地图和生命值，地图中存储着起点、终点、通路和地刺，在确保到达终点前生命值大于 0 并且步数最小。

首先对地图进行录入，接着我们要寻找起点，并将起点入队。接着是对队列的操作，在队列非空时，取出当前位于最前侧的节点。

首先进行判断，观察这个节点是否为终点。若不是终点则再进行剪枝判断，观察这个节点是否越界。若不越界，则再判断该节点时地刺还是通路。如果是通路则判断是否曾经访问过，若为访问过，则更新各个特征值并进行入队操作。如果时地刺，则还需要先判断当前的生命值是否大于 1，若大于一再将生命值减一判断该节点是否被访问过。若未访问过，则更新各个特征值然后进行入队操作。

在本题中我们运用到了结构体和队列两个数据结构。结构体用来存储一个节点的多个变量，队列实际上是一个结构体数组来存储访问过的节点信息，根据索引的变化来进行入队出队操作。

【5、AC 截图及运行测试截图】



【1、bfs-开灯关灯】

4x4灯光开关谜题的最小步数求解

描述：

给定一个4x4的灯光矩阵，每个灯的状态为开（'w'）或关（'b'）。

按下某个灯的开关会改变该灯及其上下左右相邻灯的状态（开变关，关变开）。

请编写程序，计算通过按下开关使所有灯都关闭的最少步数。

如果不存在使所有灯关闭的按法，则输出"Impossible"。

【2、程序代码及注释】

```
#include<iostream>
#include <cstring>
using namespace std;

int b[4][4]; // 定义一个4x4的整数数组b，用于存储当前灯的状态（0表示关，1表示开）
int a[4][4]; // 定义一个4x4的整数数组a，用于存储初始灯的状态
int c[4][4]; // 定义一个4x4的整数数组c，用于表示按下的开关的位置

// set函数用于模拟按下某个灯的开关，改变相应位置及其上下左右灯的状态
void set(int x, int y) {
    b[x][y] = b[x][y] ^ 1; // 将灯x,y的状态取反（0变1，1变0）
    if (x - 1 >= 0) b[x - 1][y] = b[x - 1][y] ^ 1; // 上方灯状态取反
    if (x + 1 < 4) b[x + 1][y] = b[x + 1][y] ^ 1; // 下方灯状态取反
    if (y - 1 >= 0) b[x][y - 1] = b[x][y - 1] ^ 1; // 左方灯状态取反
    if (y + 1 < 4) b[x][y + 1] = b[x][y + 1] ^ 1; // 右方灯状态取反
}

// generate函数用于根据二进制数n生成一个4位的模式，存储在数组c的第一行，表示按下第一行的哪个开关
void generate(int n) {
    for (int i = 0; i < 4; i++) {
        c[0][i] = n % 2; // 将n的最低位赋值给c[0][i]
        n = n / 2; // 右移n，为下一次循环准备
    }
}

int main() {
    int temp, min = 100000;  // temp用于临时存储步数，min用于存储
```

```
    最小步数
    bool flag = false;    // 标志变量，用于判断是否找到可行解
    char cc;      // 用于存储输入的单个字符
    int j;        // 循环变量
    // 读取输入，将字符转换为灯的开关状态
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            cin >> cc;
            if (cc == 'b')
                a[i][j] = 0; // 如果输入是'b'，则灯是关的
            else
                a[i][j] = 1; // 否则，灯是开的
        }
    }

    // 有四列，则第一行开关的按下方式有16种模式
    for (int n = 0; n < 16; n++) {
        // 从初始状态开始，复制a到b
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 4; j++) {
                b[i][j] = a[i][j];
                c[i][j] = 0; // 初始化c数组
            }
        }
        // 生成目标模式，表示各个情况下按下第一行哪个开关
        generate(n);
        temp = 0; // 重置步数计数器
        // 根据c的第一行设置第一行的灯状态
        for (j = 0; j < 4; j++) {
            if (c[0][j] == 1) {  //若按下该开关
                set(0, j);   //则对应b数组中的灯光状况发生变化
                temp++;    // 增加步数
            }
        }
        // 根据c的模式设置剩余行的灯状态
        for (int i = 1; i < 4; i++) {
            for (int j = 0; j < 4; j++) {
                if (b[i - 1][j] == 1) { // 如果上一行对应的灯是开的
                    set(i, j);   //则需要按下下一行开关
                    c[i][j] = 1;   // 标记c[i][j]为1，表示开关按下
                    temp++;    // 增加步数
```

```
                }
            }
        }
        // 检查最后一行是否所有灯都关闭
        for (j = 0; j < 4; j++) {
            if (b[3][j] == 1) break; // 如果有灯是开的，则退出循环
        }
        if (j >= 4) { // 如果所有灯都关闭，说明找到了一种可行的模式
            min = (min < temp) ? min : temp; // 更新最小步数
            flag = true; // 设置标志变量
        }
    }
    // 输出结果
    if (flag)
        cout << min << endl; // 如果找到了可行解，输出最小步数
    else
        cout << "Impossible" << endl; // 如果没有找到可行解，输出 "Impossible"
    return 0;
}
```

【3、遇到的问题及解决方法】

无。

【4、算法分析与知识点】

首先我们对题目进行分析，根据题目描述，输入一个 4*4 的灯光矩阵，要求按下最少的开关数，使得所有灯光全部开启或关闭。

首先一个按钮带来的影响是固定的，所以每个开关只会被按到一次。再次基础上，第一行开着的灯只能由第二行的开关来关闭，由此其实只要知道第一行灯光的初始情况，那么接下来所有的开关情况都可以确定了。

所以我们只需要对第一行的开关情况进行分类讨论，因为该开关矩阵一共有四列，所以所有可能的情况数为 2^4=16.我们构造了一个 generate()函数来表示按下第一行开关的 16 种情况。

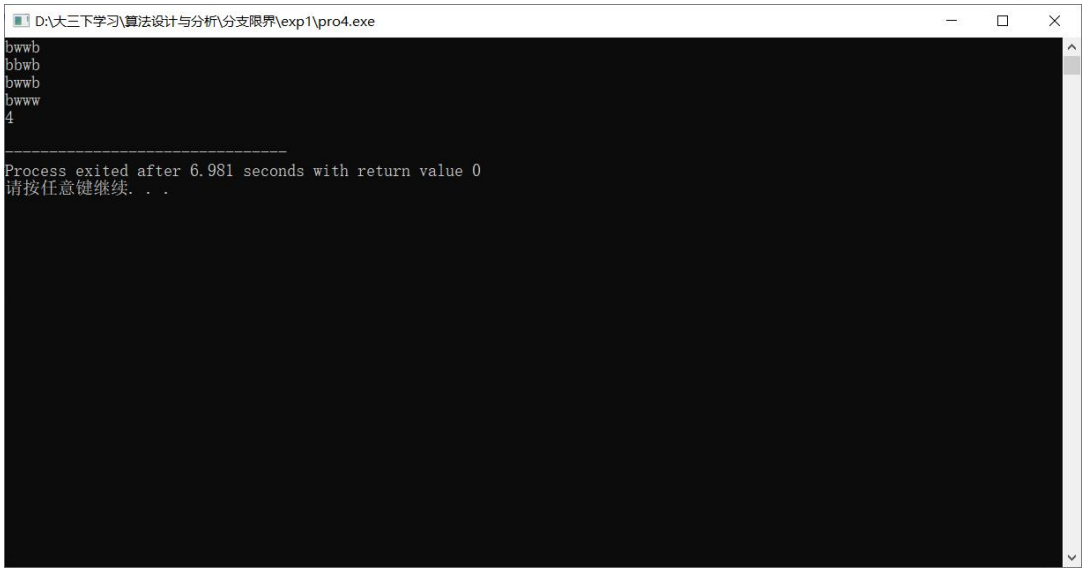
接下来只需要对各个情况进行判断计算。首先根据 generate()函数中生成的第一行按下开关的位置进行 set()操作来改变 b 矩阵中当前的灯光状况。接下来我们只要根据 b 矩阵中第一行的灯光状况，若为亮，则按下对应位置下方的开关，并更新对应的 b 矩阵的开关状况，步长和 c 矩阵。

最后当我们按完第四列开关后，判断 b 矩阵最后一列灯光是否全部关闭。若全部关闭则为一种方案，记录下 flag 为 True，并将 temp 与 min 进行对比，若比 min 小则进行更新。

要注意的是在本题中我们总共用到了三个矩阵 a,b,c。这三个矩阵分别代表了初始的灯光状况，按下开关后的灯光状况和按下开关位置的矩阵。

解答本题的关键是找到各个开关只按一次，并且只与第一行灯开关的状况有关的条件。因为要将第一行的灯的全部关闭（开启）只能由第二行的灯来操作。

【5、AC 截图及运行测试截图】



【1、bfs-极速营救】

给定一个 `n` 行 `m` 列的二维地图，地图中 `0` 表示海洋（可通行），非 `0` 表示障碍物（不可通行）。
起点坐标和终点坐标给定。
每次移动可以沿四个方向（上下左右）连续移动任意距离，直到遇到障碍物或边界。
求从起点到终点的路径，使得路径的转弯次数最少。

【2、程序代码及注释】

```
#include<iostream>
using namespace std;
int n,m;     //代表 n 行 m 列的海洋地图
int state[10001][3]; //记录每个访问状态下的特征，分别包括坐标和转弯数
int head,tail;    //用于对队列进行控制
int c,s;     //c 代表输入的图,s 代表转弯计数值。
int x,y;     //现在判断点的坐标
int x2,y2;     //终点坐标
bool ok[101][101],a[101][101];
int dx[4]={0,0,1,-1},dy[4]={1,-1,0,0};//拓展方向
void bfs()
{
 head=0;
 tail=1;
 while(head<tail)//空队退出
  {
   head++;//出队
   for (int i=0;i<4;i++)//4 个方向
   {
```

```
    x=state[head][0];y=state[head][1];
    int j=1;
    while (!a[x+dx[i]*j][y+dy[i]*j] && x+dx[i]*j<=n && y+dy[i]*j<=m && x+dx[i]*j>0 && y+dy[i]*j>0)
    //如果碰到墙或边缘就结束拓展
    {
      if (!ok[x+dx[i]*j][y+dy[i]*j])
      //如果已经走过的路线就不入队
      {
       if (x+dx[i]*j==x2 && y+dy[i]*j==y2)//结束判断
        {
         s=state[head][2];
         return;
        }
       tail++;//入队
       state[tail][0]=x+dx[i]*j;
       state[tail][1]=y+dy[i]*j;//储存位置
       state[tail][2]=state[head][2]+1;//储存转弯数
       ok[x+dx[i]*j][y+dy[i]*j]=true;//已经走过该地
      }
      j++;//下一个拓展
    }
   }
  }
}

void map_generate() { //初始化构建图例
 cin>>n>>m;
 for (int i=1;i<=n;i++)
 {
   for (int j=1;j<=m;j++)
   {
    cin>>c;
    if (c!=0) a[i][j]=true;
   }
 }
 cin>>state[1][0]>>state[1][1]>>x2>>y2;
}

int main()
{
 map_generate();
 bfs();
```

```
62.  cout<<s<<endl;
63.}
```

【3、遇到的问题及解决方法】

无。

【4、算法分析与知识点】

首先我们对题目进行分析，当船只在海洋中中航行时，从终点到目的地的转弯数要最少。这与一般的广搜算法不同的是，为了能够让转弯数最少，我们每次在前进的时候都要走到底部直到碰到障碍物为止。

我们使用广度搜索时从起点开始进行拓展，采用队列的数据结构形式。分别遍历四个方向，并在每个方向往最深处走。在每个方向往深处走时首先要判断是否越界或者碰到障碍物。接着再判断该节点是否走过，若未走过，则将该节点入队，记录坐标与转弯数。接着再增加 j 的值，往一个同方向进行扩展，直到走到终点或者碰到边界。若是碰到边界则更换方向进行上述步骤搜索。

等到所有方向全部被搜索完成后，将队列中的下一个节点入队，进行搜索。

如此进行广搜，到我们走到终点时就记录最小的拐弯数，将拐弯数存储在变量 s 中，在主函数将 s 输出即可得到答案。

【5、AC 截图及运行测试截图】

